



SQL SERVER 2025

Yazılımcılar İçin SQL Server 2025

Vektör, JSON, RegEx ve AI · görsellerle ve canlı örneklerle geliştirici rehberi

Çağlar Özenç · caglarozenc.com · DMC Bilgi Teknolojileri · 2026

Önsöz

SQL Server 2025 çıktığında en çok merak ettiğim şey, pazarlama başlıklarının arkasında bir geliştiricinin günlük işini gerçekten neyin değiştirdiği idi. Yıllardır SQL Server'la yaşayan biri olarak, "yeni sürüm geldi" cümlesinin çoğu zaman "birkaç yeni bayrak, birkaç uyarı ve bir sürü ince ayrıntı" demek olduğunu bilirim.

Bu kitabı, o ince ayrıntıları bir kez kendim yaşayıp yazmak için hazırladım. İçindeki hemen her örneği canlı bir SQL Server 2025 üzerinde çalıştırdım; dokümanla derlemenin ayrıştığı, bir özelliğin kutu üründe ve bulutta farklı davrandığı yerleri tek tek işaretledim. Amacım, aynı tuzaklara benden sonra düşmemen.

Kitap bir DBA el kitabı değil; **uygulama yazan bir mühendisin** gözünden yazıldı. Baştan sona okuyabilir ya da ihtiyacın olan bölüme atlayabilirsin; her bölüm tek başına ayakta durur. Örneklerin tamamı depoda çalıştırılabilir hâlde ve her push'ta gerçek bir SQL Server 2025 üzerinde otomatik doğrulanıyor.

İyi okumalar.

Çağlar Özenç

Kısım I · Temeller

Ortamı 60 saniyede kur, önizleme bayraklarını aç ve kitabın nasıl okunacağını gör.

1. Giriş: bu kitap kimin için

Bu e-book, **yazılım geliştiriciler, yazılım uzmanları ve SQL developer'lar** için yazıldı. Amaç, SQL Server 2025'i bir DBA gözünden değil, **uygulama yazan bir mühendisin gözünden** anlatmak: yeni T-SQL yeteneklerini, veritabanını bir AI-uygulama platformuna dönüştüren özellikleri ve bunları .NET / EF Core / Python kodundan nasıl kullanacağını.

Kitaptaki **hemen her örnek, gerçek bir SQL Server 2025 örneğinde çalıştırılıp çıktısıyla birlikte** verilmiştir. Kullanılan sürüm:

↳ **gerçek çıktı / real output**

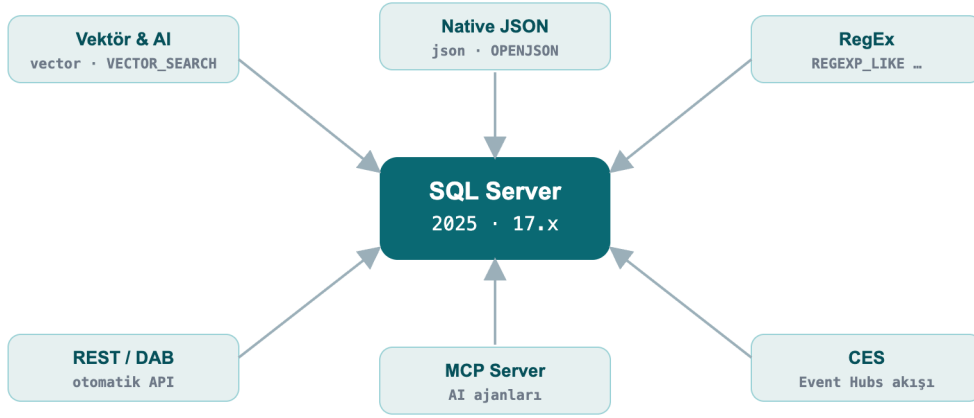
Microsoft SQL Server 2025 (RTM-CU7) – 17.0.4065.4 (X64)
Enterprise Developer Edition (64-bit) on Linux (Ubuntu 24.04)

Neden gerçek çıktı? Önizleme özelliklerinin sözdizimi dokümanla derleme (build) arasında farklılaşabilir. Bu kitap, örnekleri canlı bir motorda doğrularak "dokümanda öyle yazıyordu ama çalışmadı" tuzağını ortadan kaldırır. Nerede kutu ürünle Azure SQL/Fabric ayrıştıysa, bu ayrım açıkça belirtilmiştir.

1.1 Neler değişti: 30 saniyelik özet

SQL Server 2025 (sürüm **17.x**), 2016'dan bu yana en geniş motor sürümü. Geliştirici açısından öne çıkanlar:

- Yerleşik vektör tipi ve vektör arama:** RAG / semantik arama için ayrı bir vektör veritabanına gerek kalmadan.
- Gerçek `json` veri tipi:** `nvarchar(max)` içinde JSON saklama dönemi bitti.
- Dilin içinde RegEx:** CLR'sız desen eşleştirme, çıkarma, doğrulama.
- Uygulama entegrasyonu:** T-SQL'den REST çağrısı, otomatik REST/GraphQL (Data API Builder), AI ajanları için MCP sunucusu, Event Hubs'a değişiklik akışı (CES).
- Eşzamanlılık:** Optimized Locking blokajı ve kilit belleğini köklü biçimde azaltır.
- Ücretsiz geliştirici sürümleri:** Standard Developer ve Enterprise Developer.



Şekil 1: Veritabanı artık bir AI uygulama platformu

1.2 Ortamı Docker ile 60 saniyede kur

Bu kitabı hazırlarken kullandığım komut; sende de aynısı çalışır:

» bash

```
docker run -e "ACCEPT_EULA=Y" -e "MSSQL_SA_PASSWORD=GucLu#Parola1" \
-e "MSSQL_PID=Developer" -p 1433:1433 --name sqldev2025 -d \
mcr.microsoft.com/mssql/server:2025-latest
```

Bağlanmak için `sqlcmd` (18+), **SSMS 21**, **VS Code mssql eklentisi** veya Azure Data Studio kullanabilirsin. Konteynerin içinden hızlı test:

» bash

```
docker exec -it sqldev2025 /opt/mssql-tools18/bin/sqlcmd \
-S localhost -U sa -P 'GucLu#Parola1' -C -No -Q "SELECT @@VERSION"
```

Not: `-C -No` = sunucu sertifikasına güven + şifrelemeyi zorlama. Üretimde geçerli bir sertifika kullan; SQL Server 2025 aktarımda **TLS 1.3 + TDS 8.0**'ı destekler.

1.3 Önizleme özelliklerini ve uyumluluk düzeyini aç

Bazı yenilikler (vektör indeks, CES, fuzzy eşleşme vb.) **veritabanı-kapsamlı önizleme bayrağı** ister. Ayrıca yeni fonksiyonların bir kısmı **uyumluluk düzeyi 170**'e ihtiyaç duyar (`AI_GENERATE_CHUNKS` gibi).

» sql

```
CREATE DATABASE DevBook;
ALTER DATABASE DevBook SET COMPATIBILITY_LEVEL = 170;
GO
-- Önizleme özelliklerini aç (veritabanına kalıcı yazılır)
ALTER DATABASE SCOPED CONFIGURATION SET PREVIEW_FEATURES = ON;
```

Kontrol:

» sql

```
SELECT name, value FROM sys.database_scoped_configurations WHERE name = 'PREVIEW_FEATURES';
```

↳ gerçek çıktı / real output

name	value
-----	-----
PREVIEW_FEATURES	1

Kısım II · Yapay Zekâ ve Yeni Veri Tipleri

Vektör arama, native JSON ve dilin içindeki RegEx. Bu kısımda verinin kendisi değişiyor: veritabanı artık embedding saklıyor, belge tutuyor ve desen eşleştiriyor.

2. Yapay Zekâ ve Vektör Arama

Sürümün manşeti bu. SQL Server 2025, **embedding'leri saklar, indeksler ve arar**; hatta T-SQL içinden **embedding üretir**. Bir RAG (retrieval-augmented generation) hattını veritabanının içinde kurabilirsiniz.

2.1 `vector` veri tipi ve vektör fonksiyonları

`vector(n)` sabit boyutlu bir sayısal dizi saklar; optimize ikili formatta tutulur ama JSON dizisi gibi görünür. Eleman başına tek (4 bayt, float32) duyarlık kullanılır.

» 02-vector-funcs.sql

```
DECLARE @a vector(3) = '[1,2,3]', @b vector(3) = '[2,3,4]';
SELECT VECTOR_DISTANCE('cosine', @a, @b) AS cosine_dist,
       VECTOR_DISTANCE('euclidean', @a, @b) AS euclid_dist,
       VECTOR_NORM(@a, 'norm2') AS l2norm,
       CAST(VECTOR_NORMALIZE(@a, 'norm2') AS varchar(64)) AS normalized;
```

↳ gerçek çıktı / real output

cosine_dist	euclid_dist	l2norm	normalized
7.4167251586914062E-3	1.7320507764816284	3.7416573867739413	[2.6726124e-001,5.3452247e-001,8.0178368e-001]

`VECTOR_DISTANCE` üç metrik destekler: `'cosine'`, `'euclidean'`, `'dot'`. Vektörün özelliklerini `VECTORPROPERTY` ile okursun:

» sql

```
DECLARE @a vector(3) = '[1,2,3]';
SELECT VECTORPROPERTY(@a, 'Dimensions') AS dims, VECTORPROPERTY(@a, 'BaseType') AS basetype;
```

↳ gerçek çıktı / real output

dims	basetype
3	float32

2.2 Exact (kNN) arama: her yerde çalışan güvenli yol

En yakın komşuları bulmak için indekse gerek yok: `VECTOR_DISTANCE` + `ORDER BY` **tam (exact / kNN)** arama yapar. Küçük-orta kümelerde ve tüm sürümlerde (kutu + Azure + Fabric) çalışır.

» 03-exact-knn.sql

```
DECLARE @qv vector(5) = '[0.3,0.3,0.3,0.3,0.3]';
SELECT TOP (3) id, title, VECTOR_DISTANCE('cosine', embedding, @qv) AS dist
FROM dbo.Articles
ORDER BY dist;
```

↳ gerçek çıktı / real output

id	title	dist
15	Article 15	0.09546583890914917
30	Article 30	0.09546583890914917
49	Article 49	0.09546583890914917

Saha örneği: benzer destek biletleri. Yeni bir destek bileti geldiğinde geçmişte çözülmüş benzer biletleri bulmak çözüm süresini kısaltır. Biletlerin embedding'ini bir `vector` kolonunda tutar, yeni gelen bileti aynı modelle embed edip en yakın komşuları çekersin:

» saha-tickets.sql

```
CREATE TABLE dbo.Tickets (id int PRIMARY KEY, konu nvarchar(60), v vector(3));
INSERT dbo.Tickets VALUES
(1,'Sorgu çok yavaş çalışıyor','[0.90,0.10,0.05]'),
(2,'Uygulama veritabanına bağlanamıyor','[0.10,0.90,0.10]'),
(3,'Rapor ekranı geç açılıyor','[0.82,0.18,0.08]'),
(4,'Gece yedeği başarısız oldu','[0.08,0.12,0.90]');

-- Yeni bilet performans temalı; en benzer 2 geçmiş bilet
DECLARE @yeni vector(3) = '[0.88,0.12,0.06]';
SELECT TOP 2 id, konu, ROUND(VECTOR_DISTANCE('cosine', v, @yeni), 4) AS uzaklik
FROM dbo.Tickets ORDER BY uzaklik;
```

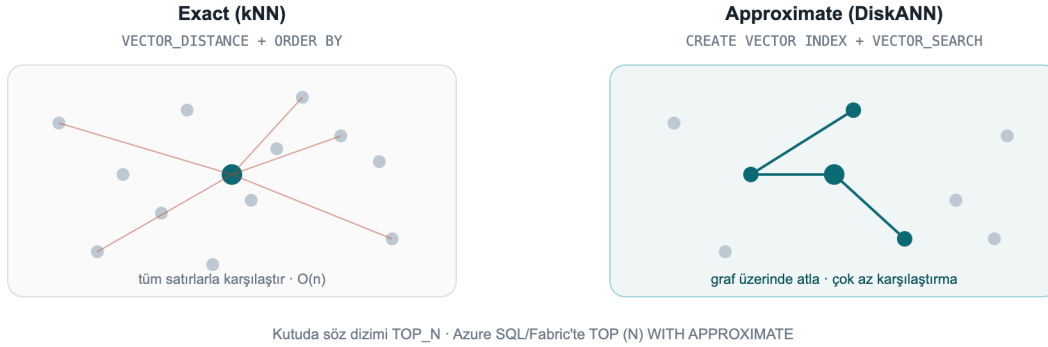
↳ gerçek çıktı / real output

id	konu	uzaklik
1	Sorgu çok yavaş çalışıyor	0.0004
3	Rapor ekranı geç açılıyor	0.0036

Performans temalı yeni bilet, bağlantı veya yedekleme biletlerini değil iki performans biletini getirdi. Gerçekte `vector(3)` yerine bir embedding modelinden gelen `vector(384)` / `vector(1536)` kullanırsın (bkz. Bölüm 11.4).

2.3 Yaklaşık (ANN) arama: DiskANN vektör indeksi

Milyonlarca satırda exact tarama pahalıdır. **DiskANN** tabanlı bir vektör indeksi, yaklaşık en yakın komşu (ANN) aramasıyla bunu ölçeklenebilir kılar.



Şekil 2: Exact (kNN) tüm satırlarla karşılaştırır; DiskANN (ANN) graf üzerinde atlayarak çok daha az karşılaştırma yapar

Denerken dört pratik tuzağa takıldım, sırayla:

Tuzak 1, Önizleme bayrağı. Vektör indeks + `VECTOR_SEARCH` kutu üründe önizlemedir: `PREVIEW_FEATURES = ON` gerekir (Azure SQL DB / Fabric SQL'de gerekmez).

Tuzak 2, `SET QUOTED\_IDENTIFIER ON`. Bu ayar kapalıysa indeks oluşturma **Msg 1934** ile başarısız olur (sqlcmd'de varsayılan kapalıdır; SSMS'te açıktır):

↳ gerçek hata: **QUOTED_IDENTIFIER** kapalıyken

Msg 1934, Level 16: CREATE VECTOR INDEX failed because the following SET options have incorrect settings: 'QUOTED_IDENTIFIER'.

Doğru kurulum:

» 04-vector-index.sql

```
SET QUOTED_IDENTIFIER ON;

-- En az 100 satır gerekir; embedding'ler örnek amaçlı üretiliyor
CREATE TABLE dbo.Articles (id int PRIMARY KEY, title nvarchar(100), embedding vector(5));
INSERT dbo.Articles (id, title, embedding)
SELECT value, 'Article ' || value,
       CAST(JSON_ARRAY(value*0.01, value*0.02, value*0.03, value*0.04, value*0.05) AS
vector(5))
FROM GENERATE_SERIES(1, 100);

CREATE VECTOR INDEX vec_idx ON dbo.Articles(embedding)
WITH (METRIC = 'cosine', TYPE = 'diskann');

SELECT name, type_desc FROM sys.vector_indexes;
```


↳ gerçek çıktı / real output

name	type_desc
vec_idx	VECTOR

Tuzak 3, Sözdizimi kutuda `TOP\_N`, Azure/Fabric'te `WITH APPROXIMATE`. Bu, testlerimde çıkan en önemli nokta. Dokümandaki yeni `SELECT TOP (N) WITH APPROXIMATE` sözdizimi "en güncel sürüm indeksler" içindir ve şu an **yalnız Azure SQL Database ile Fabric SQL**'de vardır. SQL Server 2025 kutu ürününde (CU7) bunu denersen:

↳ gerçek hata: kutu üründe WITH APPROXIMATE

Msg 102, Level 15: Incorrect syntax near 'APPROXIMATE'.

Kutu üründe `VECTOR_SEARCH` fonksiyonunu `TOP_N` parametresiyle çağırırsın.

Tuzak 4, Alias kuralı. Tablo kolonlarına **tablo alias'ıyla** (`t`), üretilen `distance` kolonuna **sonuç alias'ıyla** (`s`) erişirsin; karıştırırsan "Invalid column name" alırsın.

» 05-vector-search-box.sql

```
SET QUOTED_IDENTIFIER ON;
DECLARE @qv vector(5) = '[0.3,0.3,0.3,0.3,0.3]';

SELECT t.id, t.title, s.distance
FROM VECTOR_SEARCH(
    TABLE      = dbo.Articles AS t,    -- tablo alias'ı: t
    COLUMN      = embedding,
    SIMILAR_TO  = @qv,
    METRIC      = 'cosine',
    TOP_N       = 3) AS s              -- sonuç alias'ı: s
ORDER BY s.distance;
```

↳ gerçek çıktı / real output

id	title	distance
1	Article 1	9.5465958118438721E-2
2	Article 2	9.5465958118438721E-2
3	Article 3	9.5465958118438721E-2

Azure SQL DB / Fabric SQL'de aynı sorgu "en güncel sürüm" indeksle şöyle yazılır, `TOP_N` yerine `TOP (N) WITH APPROXIMATE`, ve `ORDER BY` yalnızca `distance ASC` olmalı: `SELECT TOP (3) WITH APPROXIMATE t.id, s.distance FROM VECTOR_SEARCH(TABLE = ... AS t, COLUMN = ..., SIMILAR_TO = @qv, METRIC = 'cosine') AS s ORDER BY s.distance;` . Güncel indeksler ayrıca **tam DML** (INSERT/UPDATE/DELETE) ve **iterative filtering** (WHERE'in aramanın içinde uygulanması) getirir; eski indeksler tabloyu salt-okunur yapıp filtreyi aramadan sonra uygulardı.

Vektör arama: kutu ürün vs bulut

SQL Server 2025 (kutu)	Azure SQL DB / Fabric
<pre>VECTOR_SEARCH(TABLE=t AS x, ..., METRIC='cosine', TOP_N = 5)</pre>	<pre>SELECT TOP (5) WITH APPROXIMATE FROM VECTOR_SEARCH(...) ORDER BY distance;</pre>

Şekil 3: Aynı 'VECTOR_SEARCH', kutuda 'TOP_N', Azure SQL/Fabric'te 'TOP (N) WITH APPROXIMATE'

2.4 Embedding'i veritabanının içinde üret

`CREATE EXTERNAL MODEL` bir AI çıkarım uç noktasını (Azure OpenAI, OpenAI, Ollama, ONNX Runtime) veritabanı nesnesi olarak tanımlar; `AI_GENERATE_EMBEDDINGS` bu modelle embedding üretir. Managed Identity ile **anahtar saklamadan** kimlik doğrularsın.

» 06-external-model.sql

```
-- Ön koşul: dış REST çağrısını aç (Azure SQL DB / Fabric'te varsayılan açık)  
EXEC sp_configure 'external rest endpoint enabled', 1; RECONFIGURE WITH OVERRIDE;  
  
-- Managed Identity tabanlı kimlik (secret alanı zorunlu)  
CREATE DATABASE SCOPED CREDENTIAL [https://my-aoai.cognitiveservices.azure.com/]  
    WITH IDENTITY = 'Managed Identity',  
    secret = '{"resourceid":"https://cognitiveservices.azure.com"}';  
  
CREATE EXTERNAL MODEL AzureEmbed WITH (  
    LOCATION = 'https://my-aoai.cognitiveservices.azure.com/openai/deployments/text-  
embedding-3-small/embeddings?api-version=2024-02-01',  
    API_FORMAT = 'Azure OpenAI',  
    MODEL_TYPE = EMBEDDINGS,  
    MODEL = 'text-embedding-3-small',  
    CREDENTIAL = [https://my-aoai.cognitiveservices.azure.com/]);
```

Not: Credential adı, LOCATION protokol + host'uyla birebir eşleşmeli. MODEL_TYPE için şu an tek geçerli değer EMBEDDINGS. API_FORMAT kabul edilenler: Azure OpenAI, OpenAI, Ollama, ONNX Runtime (ONNX ile yerel, internetsiz çalışabilirsin).

HTTPS zorunlu (bizzat doğruladım): Motor yalnızca HTTPS/TLS uç noktalara izin verir. HTTP bir Ollama uç noktası denediğimde motor net biçimde reddetti:

>

Msg 31610 ... Accessing the external endpoint is only allowed via HTTPS.

>

Yerel bir modeli (Ollama/ONNX) kullanacaksan uç noktayı **geçerli/güvenilen bir sertifikayla TLS** arkasına al; pratikte çoğu ekip `LOCATION` 'ı doğrudan **Azure OpenAI**'ye yönlendirir.

Canlı doğrulama: motor içinden embedding üretimi

`AI_GENERATE_EMBEDDINGS` 'i **iki uçtan uca senaryoda bizzat çalıştırdım**. İkisinde de motor embedding'i kendisi üretip `vector` kolonuna yazdı, sonra soruyu da aynı modelle embed edip `VECTOR_DISTANCE` ile aradı.

A) Azure OpenAI (üretim yolu). `text-embedding-3-small` (1536 boyut), kimlik `HTTPEndpointHeaders` (api-key) ile:

» 29-aoai-embed.sql

```
CREATE TABLE dbo.Kb (id int IDENTITY PRIMARY KEY, chunk nvarchar(200), emb vector(1536));
INSERT dbo.Kb (chunk, emb)
SELECT c.chunk, AI_GENERATE_EMBEDDINGS(c.chunk USE MODEL AzureEmbed)
FROM (VALUES (N'SQL Server 2025 offers a built-in vector type and DiskANN index for semantic search.'),
             (N'The native JSON type stores documents in a binary format.'),
             (N'Regular expressions provide pattern matching without CLR.')) c(chunk);

DECLARE @q vector(1536) = AI_GENERATE_EMBEDDINGS(N'how do I search text by meaning?' USE MODEL AzureEmbed);
SELECT TOP 2 id, LEFT(chunk,52) AS chunk, ROUND(VECTOR_DISTANCE('cosine', emb, @q), 4) AS dist
FROM dbo.Kb ORDER BY dist;
```

↳ gerçek çıktı / real output

id	chunk	dist
1	SQL Server 2025 offers a built-in vector type and Di	0.7059
3	Regular expressions provide pattern matching without	0.8240

B) Yerel model (Ollama), motor içinden. Modeli bilgisayarında tutmak istersen bir ayrıntı önemli: SQL Server yalnızca **güvenilir** bir HTTPS uç noktasına izin verir ve **self-signed sertifikayı kabul etmez**. Bunu ayrıntısıyla test ettim: container'ın işletim sistemi sertifikaya güvense bile (OpenSSL doğrulaması OK), motorun REST istemcisi güvenmiyor ve isteği hiç göndermeden `0x80070008` veriyor. Çözüm: Ollama'yı bilgisayarında çalıştırıp önüne **güvenilir sertifikalı bir tünel** (ör. ücretsiz `cloudflared`) koy. Model tamamen yerel kalır; SQL yalnızca tünelin güvenilir sertifikasını görür:

» 30-local-ollama-embed.sql

```
-- Terminalde: ollama serve + cloudflared tunnel --url http://localhost:11434
CREATE EXTERNAL MODEL LocalOllama WITH (
  LOCATION = 'https://<tunel-adi>.trycloudflare.com/api/embed',
  API_FORMAT = 'Ollama', MODEL_TYPE = EMBEDDINGS, MODEL = 'all-minilm');

DECLARE @q vector(384) = AI_GENERATE_EMBEDDINGS(N'how do I find similar text by meaning?' USE
MODEL LocalOllama);
SELECT TOP 2 id, LEFT(chunk,46) AS chunk, ROUND(VECTOR_DISTANCE('cosine', emb, @q), 4) AS dist
FROM dbo.KbLocal ORDER BY dist;
```

↳ gerçek çıktı / real output

id	chunk	dist
1	SQL Server 2025 has a built-in vector type and	0.6912
3	Regular expressions match patterns without CLR	0.7352

Her iki senaryoda da embedding'i motor kendisi üretti ve anlamsal arama yerel SQL Server 2025 üzerinde çalıştı. Yerel yol için gereken tek ek bileşen, self-signed sertifikayı SQL kabul etmediği için güvenilir sertifikalı tüneldir. (Bölüm 11.4'teki demo ise embedding'leri istemci tarafında üretim yükler; tünel bile gerekmez.)

2.5 Metni parçalara böl: AI_GENERATE_CHUNKS

RAG için uzun metni modele sığacak parçalara bölmen gerekir. `AI_GENERATE_CHUNKS` bunu yapan bir **tablo-değerli fonksiyondur**, yani `CROSS APPLY` ile kullanılır ve `chunk`, `chunk_order`, `chunk_offset`, `chunk_length` kolonları döndürür. **Harici uç nokta gerektirmez**, bu yüzden aşağıdaki çıktı gerçek:

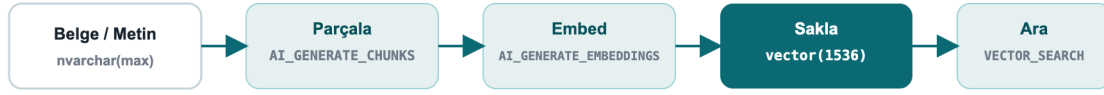
» 07-chunks.sql

```
SELECT c.chunk, c.chunk_order, c.chunk_length
FROM (VALUES (N'SQL Server 2025 yazılımcılar için yerleşik vektör ve JSON getirir.')) d(t)
CROSS APPLY AI_GENERATE_CHUNKS(SOURCE = d.t, CHUNK_TYPE = FIXED, CHUNK_SIZE = 25, OVERLAP = 0)
AS c;
```

↳ gerçek çıktı / real output

chunk	chunk_order	chunk_length
SQL Server 2025 yazılımcı	1	25
lar için yerleşik vektör	2	25
ve JSON getirir.	3	16

`OVERLAP` (0-50 arası yüzde) ile ardışık parçaların örtüşmesini sağlayarak retrieval doğruluğunu artırırın. Tam RAG akışı: `AI_GENERATE_CHUNKS` → `AI_GENERATE_EMBEDDINGS` → `vector(1536)` kolonuna `INSERT` → sorgu vektörüyle `VECTOR_DISTANCE` / `VECTOR_SEARCH`.



Kullanıcı sorusu → embedding → en yakın parçalar → LLM'e bağlam (RAG)

Şekil 4: Uçtan uca RAG hattı, tamamen veritabanının içinde

2.6 Uygulama tarafı: .NET ve EF Core

Native vektör, `Microsoft.Data.SqlClient` 6.1.0+ ile TDS üzerinden ikili taşınır; `SqlVector<float>` tipi eklendi. .NET 10'da `SqlDbType.Vector` numaralandırması var. Eski istemciler için SQL Server vektörü `varchar(max)` olarak sunarak geriye uyumluluğu korur.

EF Core 10 vektörü ve `VECTOR_DISTANCE` 'ı tam destekler (yalnız SQL Server 2025+):

» 08-efcore.cs

```
public class Article
{
    public int Id { get; set; }
    public string Title { get; set; } = "";
    [Column(TypeName = "vector(1536)")]
    public SqlVector<float> Embedding { get; set; }
}
```

`EF.Functions.VectorDistance(...)` ile LINQ üzerinden semantik arama yaparsın; tam bağlantı dizesi, sorgu ve Python örnekleri **Bölüm 8**'de.

Not: EF Core 11'den itibaren vektör özellikleri, büyük oldukları için sorguda **varsayılan yüklenmez**, gerektiğinde açıkça seçilir. Dapper ile de `SqlVector<float>` doğrudan parametre olarak bağlanır.

2.7 Exact mı, ANN mı?: canlı ölçüm

Karar için gerçek sayı gerekir. **100.000 satır, 16 boyutlu** bir tabloda aynı sorguyu exact (indeks yok) ve DiskANN ANN (`VECTOR_SEARCH`) ile 5'er kez çalıştırdım (warm cache):



Şekil 5: Exact kNN ~13 ms, DiskANN ANN ~5 ms (100.000 satır, 16 boyut; kendi ölçümüm)

- **Exact (kNN):** ~11-14 ms (medyan ~13 ms). Her sorgu tüm satırlarla karşılaştırır: $O(n)$.
- **ANN (DiskANN):** ~4-5 ms (kimi warm çağrı <1 ms). Graf üzerinde atlar.

Bu ölçekte ANN **~3x hızlı**; asıl kazanç ise **satır sayısı ve boyut arttıkça** ortaya çıkar (milyonlarca satır ve 768/1536 boyutta fark çok daha belirginleşir). **Pratik kural:** birkaç yüz bin satıra ve düşük gecikme ihtiyacına kadar exact yeterli ve basittir; ötesinde DiskANN indeksine geç. ANN *yaklaşıktır*, %100 kesinlik şartsa exact kullan.

Kutu ürün notu: Bu ANN ölçümü kutu üründe `VECTOR_SEARCH ... TOP_N` sözdizimiyle alındı; `PREVIEW_FEATURES = ON` ve `SET QUOTED_IDENTIFIER ON` gerekir (bkz. Bölüm 10, Sorun Giderme).

3. Native JSON

Artık gerçek bir `json` veri tipi var: ikili depolanır, indekslenebilir, 2 GB'a kadar belge. `nvarchar(max)` içinde JSON tutmaya göre okuma performansı belirgin artar ve şema motor tarafından doğrulanır.

3.1 `json` kolonu oluştur ve sorgula

» 09-json-table.sql

```
CREATE TABLE dbo.Orders (Id int IDENTITY PRIMARY KEY, Doc json);
INSERT dbo.Orders (Doc) VALUES
  (N'{"customer":"Ada","total":1290.50,"items":["ssd","ram"],"paid":true}'),
  (N'{"customer":"Linus","total":540.00,"items":["kbd"],"paid":false}');

SELECT Id,
       JSON_VALUE(Doc, '$.customer') AS customer,
       CAST(JSON_VALUE(Doc, '$.total') AS decimal(10,2)) AS total,
       JSON_QUERY(Doc, '$.items') AS items,
       ISJSON(Doc) AS is_valid
FROM dbo.Orders;
```

↳ gerçek çıktı / real output

Id	customer	total	items	is_valid
1	Ada	1290.50	["ssd","ram"]	1
2	Linus	540.00	["kbd"]	1

`JSON_VALUE` skaler değer, `JSON_QUERY` nesne/dizi döndürür. `ISJSON` bir metnin geçerli JSON olup olmadığını sınar.

3.2 Belgeyi güncelle: `JSON_MODIFY`

» 10-json-modify.sql

```
UPDATE dbo.Orders SET Doc = JSON_MODIFY(Doc, '$.paid', 'true') WHERE Id = 2;
SELECT Id, JSON_VALUE(Doc, '$.paid') AS paid FROM dbo.Orders WHERE Id = 2;
```

↳ gerçek çıktı / real output

Id	paid
2	true

Nüans: `JSON_MODIFY` 'nin değer argümanı bir T-SQL metniyse JSON'a **string** olarak yazılır:

`JSON_MODIFY(Doc, '$.paid', 'true')` → `{"paid":"true"}`. Gerçek JSON boolean için değeri uygun tipte ver: `JSON_MODIFY(Doc, '$.paid', CAST(1 AS bit))` → `{"paid":true}` (canlı doğruladım).

3.3 Satırları JSON'a topla: `JSON_ARRAYAGG` / `JSON_OBJECTAGG`

2025'in iki yeni birinci sınıf agregası, string birleştirme hilelerini bitirir:

» 11-json-agg.sql

```
SELECT JSON_ARRAYAGG(JSON_VALUE(Doc, '$.customer')) AS customers,
       JSON_OBJECTAGG(JSON_VALUE(Doc, '$.customer')
                      : CAST(JSON_VALUE(Doc, '$.total') AS decimal(10,2))) AS totals
FROM dbo.Orders;
```

↳ gerçek çıktı / real output

customers	totals
["Ada","Linus"]	{"Ada":1290.50,"Linus":540.00}

3.4 JSON dizisini satırlara aç: OPENJSON

» 12-openjson.sql

```
SELECT o.Id, j.[value] AS item
FROM dbo.Orders o CROSS APPLY OPENJSON(o.Doc, '$.items') j;
```

↳ gerçek çıktı / real output

Id	item
1	ssd
1	ram
2	kbd

Saha örneği: belirli ürünü içeren siparişler. json dizisinde bir değeri aramak için OPENJSON'ı alt sorguyla kullanırsın; ayrı bir arama servisine gerek yok:

» saha-json.sql

```
SELECT Id, JSON_VALUE(Doc, '$.customer') AS musteri
FROM dbo.Orders
WHERE 'ssd' IN (SELECT value FROM OPENJSON(Doc, '$.items'));
```

↳ gerçek çıktı / real output

Id	musteri
1	Ada

4. Regular Expressions ve String Fonksiyonları

CLR'sız, dilin içinde regex nihayet geldi. Uygulama katmanındaki doğrulama/temizleme mantığının çoğunu motora indirebilirsin.

4.1 Çıkar, değiştir, say

» 13-regex.sql

```
SELECT REGEXP_REPLACE(' çok fazla boşluk ', '\s+', ' ') AS normalized,
       REGEXP_SUBSTR('SKU: ABC-12345 stok', '[A-Z]{3}-\d{5}') AS sku,
       REGEXP_COUNT('a1b2c3d4', '\d') AS digit_count,
       REGEXP_INSTR('order-2026', '\d{4}') AS year_pos;
```


↳ gerçek çıktı / real output

normalized	sku	digit_count	year_pos
çok fazla boşluk	ABC-12345	4	7

Tam liste: `REGEXP_LIKE` , `REGEXP_REPLACE` , `REGEXP_SUBSTR` , `REGEXP_INSTR` , `REGEXP_COUNT` , `REGEXP_MATCHES` , `REGEXP_SPLIT_TO_TABLE` .

4.2 Kritik nüans: `REGEXP_LIKE` bir YÜKLEMDİR

Testlerimde çıkan en önemli developer detayı: `REGEXP_LIKE` bir **arama yüklemidir (predicate)**, `WHERE` , `CASE` , `CHECK` içinde kullanılır. Onu doğrudan `SELECT ... AS ok` gibi seçemez veya `= 0` ile karşılaştıramazsın. Şu yanlış:

↳ gerçek hata: yüklem skaler gibi kullanılınca

```
Msg 102, Level 15: Incorrect syntax near '='.  -- WHERE REGEXP_LIKE(...) = 0  YANLIŞ
```

Doğru kullanım, geçersiz e-postaları bulmak için `NOT` :

» 14-regex-predicate.sql

```
SELECT email
FROM (VALUES ('gecerli@x.com'), ('gecersiz@@')) c(email)
WHERE NOT REGEXP_LIKE(email, '^[\\w.+]+@[\\w-]+\\.([\\w.-]+)?$');
```

↳ gerçek çıktı / real output

email
gecersiz@@

Skaler bir sonuç istiyorsan `CASE` sar:

» sql

```
SELECT email,
       CASE WHEN REGEXP_LIKE(email, '^[\\w.+]+@[\\w-]+\\.([\\w.-]+)?$') THEN 'valid' ELSE 'INVALID'
       END AS status
FROM (VALUES ('ada@dmc.com.tr'), ('bad@x'), ('linus@kernel.org')) c(email);
```

↳ gerçek çıktı / real output

email	status
ada@dmc.com.tr	valid
bad@x	INVALID
linus@kernel.org	valid

Saha örneği: TR telefon ve IBAN doğrulama. Form verisini uygulamaya güvenmeden, motor düzeyinde doğrularsın:

» saha-regex.sql

```
SELECT deger,  
       CASE WHEN REGEXP_LIKE(deger, '^\\+90 ?\\d{3} ?\\d{3} ?\\d{2} ?\\d{2}$') THEN 'telefon OK'  
            WHEN REGEXP_LIKE(deger, '^TR\\d{24}$') THEN 'IBAN OK'  
            ELSE 'geçersiz' END AS sonuc  
FROM (VALUES ('+90 212 945 61 66'), ('TR330006100519786457841326'), ('bozuk-veri')) v(deger);
```

↳ gerçek çıktı / real output

deger	sonuc
+90 212 945 61 66	telefon OK
TR330006100519786457841326	IBAN OK
bozuk-veri	geçersiz

4.3 Desenle tabloya böl

» 15-regex-split.sql

```
SELECT value FROM REGEXP_SPLIT_TO_TABLE('sql,server;2025|dev', '[,;|]');
```

↳ gerçek çıktı / real output

value
sql
server
2025
dev

4.4 Fuzzy (yaklaşık) string eşleşme

Yazım hatası toleransı, kayıt eşleştirme (record matching) için. Önizleme özellikleridir:

» 16-fuzzy.sql

```
SELECT EDIT_DISTANCE('İstanbul','İstambul') AS edit_dist,  
       EDIT_DISTANCE_SIMILARITY('İstanbul','İstambul') AS edit_sim,  
       JARO_WINKLER_SIMILARITY('Ankara','Ankraa') AS jw_sim;
```

↳ gerçek çıktı / real output

edit_dist	edit_sim	jw_sim
1	88	96

EDIT_DISTANCE = gereken düzenleme (ekleme/silme/değiştirme) sayısı; ***_SIMILARITY** fonksiyonları 0-100 arası benzerlik döndürür.

4.5 Uzun süredir istenen küçük eklentiler

» 17-newfuncs.sql

```
SELECT 'sql' || '-' || '2025' AS pipe_concat, -- || birleştirme
       GREATEST(3,9,4,7) AS greatest_v,
       LEAST(3,9,4,7) AS least_v,
       CURRENT_DATE AS today,
       BASE64_ENCODE(CAST('DMC' AS varbinary(8))) AS b64,
       CAST(BASE64_DECODE('RE1D') AS varchar(8)) AS b64_back;
```

↳ gerçek çıktı / real output

pipe_concat	greatest_v	least_v	today	b64	b64_back
sql-2025	9	3	2026-07-26	RE1D	DMC

`PRODUCT()` , bir kümenin çarpımını hesaplayan yeni agrega (bileşik büyüme, olasılık vb.):

» 18-product.sql

```
SELECT category, PRODUCT(factor) AS compound
FROM (VALUES ('growth',1.10), ('growth',1.05), ('growth',1.20)) t(category,factor)
GROUP BY category;
```

↳ gerçek çıktı / real output

category	compound
growth	1.386000

Ayrıca: `UNISTR` (Unicode kaçış), `CURRENT_DATE` , `SUBSTRING` 'de `length` opsiyonel (ANSI uyumu), `DATEADD` artık `bigint` kabul eder, ve `GENERATE_SERIES` ile sayı dizileri:

» sql

```
SELECT STRING_AGG(CAST(value AS varchar(4)), ',') AS series FROM GENERATE_SERIES(1, 10, 2);
```

↳ gerçek çıktı / real output

series
1,3,5,7,9

Kısım III · Entegrasyon, Performans ve Güvenlik

Veritabanını servislere ve olay akışlarına bağla, eşzamanlılığı ölçekle, veriyi katman katman koru. Üretimde asıl fark burada açılır.

5. Uygulama Entegrasyonu

SQL Server 2025, veritabanını olay-güdümlü ve servis-odaklı mimarilerin birinci sınıf bir parçası yapar.

5.1 T-SQL'den REST çağrısı: `sp_invoke_external_rest_endpoint`

Veritabanından doğrudan bir REST/GraphQL uç noktasına çağrı yap: Azure Function tetikle, Power BI güncelle, Azure OpenAI ile konuş, on-prem servis çağır.

» 19-rest.sql

```
DECLARE @response nvarchar(max);
EXEC sp_invoke_external_rest_endpoint
    @url      = 'https://api.example.com/v1/notify',
    @method   = 'POST',
    @payload  = N'{"orderId": 42, "status": "shipped"}',
    @headers  = '{"Content-Type":"application/json"}',
    @response = @response OUTPUT;
SELECT JSON_VALUE(@response, '$.result') AS result;
```

Güvenlik: Uç nokta HTTPS + TLS olmalı; kimlik bilgileri `DATABASE SCOPED CREDENTIAL` içinde tutulur. Bu özellik `sp_configure 'external rest endpoint enabled'` ile açılır.

5.2 Otomatik REST/GraphQL: Data API Builder (DAB)

Data API Builder, tablo ve view'larından **kod yazmadan** güvenli REST ve GraphQL uç noktaları üretir. Bir JSON yapılandırması yeterli:

```
{
  "data-source": { "database-type": "mssql", "connection-string": "@env('SQL_CONN')" },
  "entities": {
    "Article": {
      "source": "dbo.Articles",
      "permissions": [
        { "role": "anonymous", "actions": ["read"] },
        { "role": "authenticated", "actions": ["create", "update"] }
      ]
    }
  }
}
```

Çalıştır: `dab start`. Artık `GET /api/Article` (REST) ve `/graphql` (GraphQL) hazır, sayfalama, filtreleme, satır düzeyi güvenlik dahil. Frontend ekipleri için ideal. DAB'ı container'daki DevBook'a karşı çalıştırdım; `GET /api/Article?$first=2` gerçek yanıtı (vektör kolonu dahil):

↳ DAB gerçek yanıtı: hiç kod yazmadan

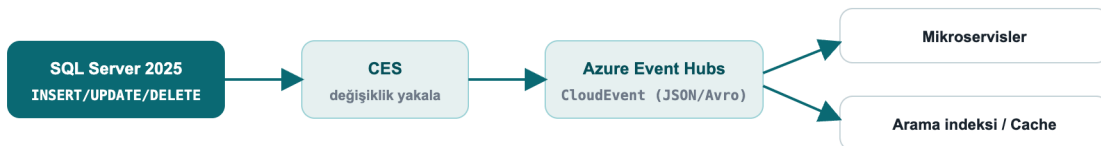
```
{ "value": [
  { "id": 1, "title": "Article 1", "embedding": [0.01, 0.02, 0.03, 0.04, 0.05] },
  { "id": 2, "title": "Article 2", "embedding": [0.02, 0.04, 0.06, 0.08, 0.10] } ] }
```

5.3 AI ajanları için: SQL MCP Server

SQL MCP Server (Data API Builder üzerinden), özel veya Foundry AI ajanlarının veritabanına **güvenli, denetimli** biçimde bağlanmasını sağlar. GitHub Copilot ve Cursor gibi araçlar MCP ile şemayı ve veriyi mantık yürüterek okuyabilir. Azure SQL Hyperscale'de doğrudan bir SQL MCP uç noktası da vardır.

5.4 Olay akışı: Change Event Streaming (CES)

CES, DML değişikliklerini (insert/update/delete) **şema + eski/yeni değerlerle** near-real-time olarak **Azure Event Hubs**'a yayımlar. Format: **CloudEvent** (JSON veya Avro Binary). CDC'nin olay-güdümlü halefi, mikroservisleri, cache invalidasyonunu, arama indekslemesini veriye senkron tutmak için.



Şekil 6: Değişiklik akışı DML → CES → Event Hubs (CloudEvent) → tüketiciler

Saha örneği: aramayı ve önbelleği veriye senkron tut. Bir sipariş güncellendiğinde CES bu değişikliği Event Hubs'a bir CloudEvent olarak yayımlar; tüketici bir mikroservis olayı alıp arama indeksini ve önbelleği

tazeler. Böylece "veritabanı yazıldı ama arama sonuçları/önbellek eski" sorununu, poll eden bir job veya uygulama tarafında ek kod yazmadan çözersin.

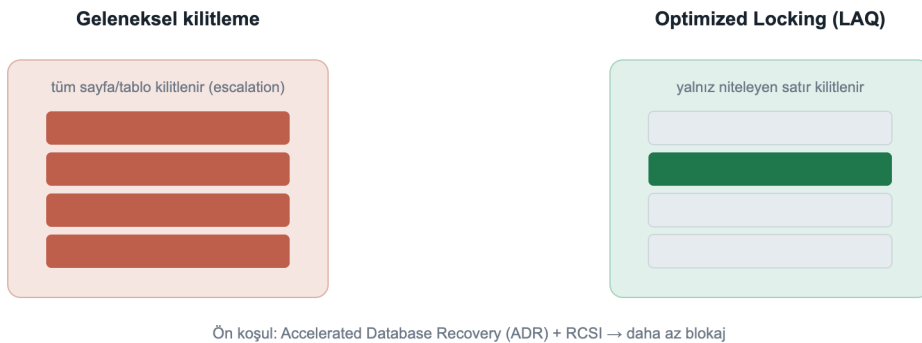
CES bir **önizleme** özelliğidir (`PREVIEW_FEATURES`). Yükseltme notu: eski **Synapse Link** kaldırıldı; analitik replikasyon için yeni yol **Fabric Mirroring**'dir (bkz. Bölüm 8).

6. Eşzamanlılık ve Performans

Developer'ın günlük hayatını en çok etkileyen motor yenilikleri.

6.1 Optimized Locking

Optimized Locking, kilit belleğini ve blokajı köklü biçimde azaltır. İki mekanizma: **Lock After Qualification (LAQ)** (satır ancak niteledikten sonra kilitlenir) ve **işlem-kimliği (TID) kilitleme**. Sonuç: kilit yükseltmesi (lock escalation) büyük ölçüde ortadan kalkar, yüksek eşzamanlı OLTP'de çok daha az blokaj.



Şekil 7: Geleneksel kilitleme tüm sayfayı/tabloyu kilitler; Optimized Locking yalnız niteleyen satırı kilitler

Ön koşul **Accelerated Database Recovery (ADR)**'dir; **RCSI** ile en iyi sonucu verir:

» `21-optimized-locking.sql`

```
ALTER DATABASE DevBook SET ACCELERATED_DATABASE_RECOVERY = ON;
ALTER DATABASE DevBook SET READ_COMMITTED_SNAPSHOT ON WITH ROLLBACK IMMEDIATE;

SELECT is_accelerated_database_recovery_on AS adr,
       is_read_committed_snapshot_on      AS rcsi
FROM sys.databases WHERE name = 'DevBook';
```

↳ gerçek çıktı / real output

```
adr  rcsi
---  ----
1    1
```

Saha örneği: kampanya anında sipariş sayacı. İndirim başladığında binlerce eşzamanlı sipariş, ortak bir stok/sayaç satırını günceller. Klasik kilitleme sayfa/tablo escalation'ına gidip blokajı patlatır. ADR + optimized locking ile yalnız niteleyen satır kilitlendiğinden throughput belirgin artar; en sıcak sayaçları In-Memory OLTP'ye (Bölüm 6.5) taşımak daha da hızlandırır.

6.2 Optional Parameter Plan Optimization (OPPO)

Klasik "catch-all" sorgu belasına (opsiyonel/NULL parametreler tek plana sıkıştır) yeni ilaç. OPPO, Parameter Sensitive Plan Optimization (PSPO) altyapısını genişleterek **opsiyonel parametre değerlerine göre farklı planlar** üretir, `WHERE (@x IS NULL OR col = @x)` kalıbındaki performans çöküşlerini azaltır.

6.3 Intelligent Query Processing (IQP) ailesi

Minimum eforla mevcut iş yükünü hızlandıran özellikler. 2025 eklentileri:

Özellik	Ne yapar
CE feedback for expressions	Kardinalite tahmini ifade düzeyinde geçmişten öğrenir
OPPO	Opsiyonel parametreye göre çoklu plan
DOP feedback (varsayılan açık)	Paralellik derecesini geçmişe göre ayarlar
Query Store for secondaries (varsayılan açık)	Okunabilir ikincillerde de Query Store
ABORT_QUERY_EXECUTION ipucu	Bilinen problemli sorgunun gelecekteki çalışmasını bloke eder

6.4 Query Store ile regresyonu yakala

Query Store, plan geçmişini ve çalışma istatistiklerini saklar; regresyona uğramış sorguyu bulup iyi planı zorlayabilirsin.

» 22-query-store.sql

```
SELECT TOP 10 q.query_id, rs.avg_duration, rs.count_executions, p.plan_id
FROM sys.query_store_runtime_stats rs
JOIN sys.query_store_plan p ON p.plan_id = rs.plan_id
JOIN sys.query_store_query q ON q.query_id = p.query_id
ORDER BY rs.avg_duration DESC;

-- iyi planı zorla
EXEC sys.sp_query_store_force_plan @query_id = 42, @plan_id = 7;
```

6.5 In-Memory OLTP: sıcak yol için bellek-optimize tablolar

Çok yüksek yazma hızı gereken "sıcak" tablolar (sayaç, oturma, kuyruk) için **bellek-optimize tablolar** kiltsiz/latch'siz çalışır. Önce bir `MEMORY_OPTIMIZED_DATA` dosya grubu gerekir:

```

ALTER DATABASE DevBook ADD FILEGROUP imoltp CONTAINS MEMORY_OPTIMIZED_DATA;
ALTER DATABASE DevBook ADD FILE (name='imoltp1', filename='/var/opt/mssql/data/imoltp1') TO
FILEGROUP imoltp;

CREATE TABLE dbo.HotCounter (Id int PRIMARY KEY NONCLUSTERED, Hits bigint)
WITH (MEMORY_OPTIMIZED = ON, DURABILITY = SCHEMA_AND_DATA);
INSERT dbo.HotCounter VALUES (1, 0);
UPDATE dbo.HotCounter SET Hits = Hits + 1 WHERE Id = 1;
SELECT name, is_memory_optimized, durability_desc FROM sys.tables WHERE name='HotCounter';

```

↳ gerçek çıktı / real output

name	is_memory_optimized	durability_desc
HotCounter	1	SCHEMA_AND_DATA

`DURABILITY = SCHEMA_AND_DATA` verinin kalıcı olmasını sağlar; `SCHEMA_ONLY` ise yalnız şemayı tutar (yeniden başlatınca veri gider, geçici/oturum verisi için ideal, en hızlısı).

7. Güvenlik (Developer Sorumluluğu)

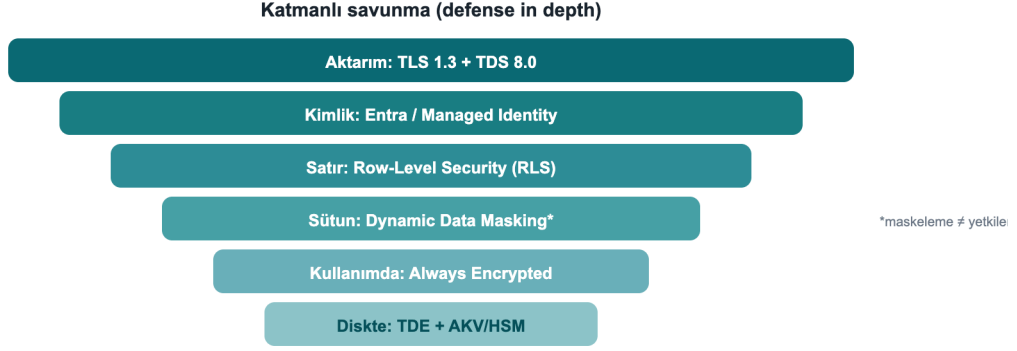
Güvenlik yalnız DBA'in değil, kod yazanın da işi. SQL Server 2025'te bir developer'ın bilmesi gerekenler.

7.1 Her zaman parametrele: SQL Injection

Yeni RegEx/JSON fonksiyonları bile string birleştirmeyele sorgu kurmayı mazur göstermez. Daima parametrelili komut kullan (`SqlParameter`, EF Core, Dapper otomatik parametreler). Dinamik SQL şartsa `sp_executesql` + parametre.

7.2 Veriyi katman katman koru

Katman	Araç	Ne zaman
Kullanımda şifreleme	Always Encrypted (+ secure enclaves)	DBA bile göremesin; enclave ile eşitlik dışı sorgu
Satır izolasyonu	Row-Level Security (RLS)	Çok-kiracılı uygulamalarda kiracı ayrımı
Maskleme	Dynamic Data Masking (DDM)	PII'yi arayüzde gizle
Kurcalama kanıtı	Ledger	Kriptografik doğrulanabilirlik / uyumluluk
Kütle şifreleme	TDE + AKV/Managed HSM	Diskte şifreli veri



Şekil 8: Katmanlı savunma: aktarımdan diske kadar her katman ayrı bir güvenceyi ekler

7.3 RLS + DDM: canlı örnek ve önemli bir uyarı

» 23-rls-ddm.sql

```
CREATE TABLE dbo.Customers (  
    Id          int,  
    TenantId int,  
    Tckn        char(11) MASKED WITH (FUNCTION = 'partial(0,"*****",2)')  
);  
INSERT dbo.Customers VALUES (1,10,'12345678901'), (2,20,'98765432109');  
SELECT Id, TenantId, Tckn FROM dbo.Customers; -- yetkili (UNMASK) kullanıcı görünümü
```

↳ gerçek çıktı / real output

Id	TenantId	Tckn
1	10	12345678901
2	20	98765432109

Maskeleye ≠ yetkilendirme. Yukarıda sa (UNMASK iznine sahip) gerçek TCKN'yi görür; maskeleye yalnız **görüntüyü** gizler ve WHERE / ORDER BY ile çıkarmaya açıktır. Gerçek gizlilik sınırı için **RLS + Always Encrypted** gerekir. DDM'yi "güvenlik" değil, "gözden kaçırma azaltıcı" olarak düşün.

7.4 2025 güvenlik güncellemeleri

- **PBKDF2 parola hash'i varsayılan** → NIST SP 800-63b uyumu.
- **TLS 1.3 + TDS 8.0:** bağlantı dizisinde `Encrypt=Strict` ile zorunlu kıl.
- **Managed Identity** (Azure Arc) ile giden bağlantılarda anahtarsız kimlik; **EKM** artık **Managed HSM**'i destekler.
- **Purview access policies kaldırıldı** → yerine sabit sunucu rolleri (`##MS_DatabaseConnector##` vb.).

7.5 Ledger: silinemez, değiştirilemez denetim kaydı

Denetim/uyumluluk kayıtlarının **kurcalanmadığını** kriptografik olarak kanıtlaman gerekiyorsa Ledger tam bunun içindir. **Append-only ledger** tablosu yalnız `INSERT` kabul eder; `UPDATE` / `DELETE` motor tarafından reddedilir:

» 28-ledger.sql

```
CREATE TABLE dbo.AuditLog (Id int IDENTITY PRIMARY KEY, Actor sysname, Action nvarchar(50))
    WITH (LEDGER = ON (APPEND_ONLY = ON));
INSERT dbo.AuditLog (Actor, Action) VALUES ('ada','GRANT'), ('linus','REVOKE');
UPDATE dbo.AuditLog SET Action = 'X' WHERE Id = 1;  -- reddedilir
```

↳ **UPDATE denemesinin gerçek sonucu**

```
Msg 37359, Level 16: Updates are not allowed for the append only Ledger table 'dbo.AuditLog'.
```

Güncellenebilir (updatable) ledger tabloları da vardır, bunlar system-versioning + otomatik ledger görünümü ile geçmiş tutar; her satırın kriptografik hash'i `sys.database_ledger_transactions` ile doğrulanabilir. Append-only ise sızıntı/silme koruması gereken **yalnız-ekleme** denetim akışları için en yalın seçenektir.

Ledger'ı uygulama tarafında değil **veritabanında** tutmak, denetim kaydının uygulama hatası veya kötü niyetli erişimle bozulmasını da engeller.

Saha örneği: KVKK/BDDK denetim izi. "Kim, ne zaman, hangi veriye erişti" kaydının sonradan değiştirilmediğini kanıtlaman gerekir. Append-only ledger tablosu bu izi tutar; bir denetçi `sys.database_ledger_transactions` üzerinden kayıtların kriptografik bütünlüğünü doğrulayabilir. Kayıt uygulama katmanında değil, motorda güvence altına alınır.

7.6 Always Encrypted: veritabanı yöneticisi bile göremez (canlı, yerel)

Always Encrypted ile veri **istemci tarafında** şifrelenir; sunucu, DBA dahil, yalnızca ciphertext görür ve anahtar sunucuda hiç bulunmaz. Bunu **tamamen yerel** kurdum (Azure ya da Windows sertifika deposu olmadan): Column Master Key olarak self-signed bir RSA sertifikası, `Microsoft.Data.SqlClient` 'ta ise **özel bir anahtar-deposu sağlayıcısı** (`SqlColumnEncryptionKeyStoreProvider`) kullandım. Derlenip çalışan tam kod: `samples/AlwaysEncryptedDemo`.

» 31-always-encrypted.sql

```
CREATE COLUMN MASTER KEY CMK1 WITH (KEY_STORE_PROVIDER_NAME='DMC_LOCAL', KEY_PATH='local/dmc-cmk');
CREATE COLUMN ENCRYPTION KEY CEK1 WITH VALUES
    (COLUMN_MASTER_KEY=CMK1, ALGORITHM='RSA_OAEP', ENCRYPTED_VALUE=0x...);  -- istemci üretir

CREATE TABLE dbo.Patients (
    Id int IDENTITY PRIMARY KEY, Name nvarchar(60),
    Ssn nvarchar(16) COLLATE Latin1_General_BIN2
    ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY=CEK1, ENCRYPTION_TYPE=DETERMINISTIC,
        ALGORITHM='AEAD_AES_256_CBC_HMAC_SHA_256'));

```

Bağlantı dizesine `Column Encryption Setting=Enabled` ekleyip sağlayıcıyı kaydeden **istemci**, düz metni görür (bizzat çalıştırdım):

↳ **istemci (AE etkin): deşifre**

Id	Name	Ssn
1	Ada Lovelace	12345678901
2	Linus T.	98765432109

Aynı satırlar, anahtarı olmayan **düz bir sorguda** (tipik DBA erişimi) yalnızca ciphertext'tir:

↳ **düz sqlcmd (anahtar yok): ciphertext**

Id	Name	Ssn
1	Ada Lovelace	0x01BB7ED4FA4BEF8EC7F92F62643801121A131A...
2	Linus T.	0x013C9A5AF77F693B2C08E79892914AAC4E68021...

Katalog da doğrular: `Ssn` sütunu DETERMINISTIC şifreli, `CEK1 / CMK1` ile, sağlayıcı `DMC_LOCAL`.

Deterministic şifreleme eşitlik araması ve join'e izin verir (metin sütununda `BIN2` collation şarttır); en yüksek gizlilik için RANDOMIZED kullan. **DDM'den farkı (Bölüm 7.3):** maskeleye yalnız görüntüyü gizler; Always Encrypted'te anahtar istemcide olduğundan sunucu veriyi hiçbir koşulda göremez.

Kısım IV · Kod, Üretim ve Pratik

Koddan bağlan, güvenle yükselt, hataları hızlı çöz ve gerçek dünya tarifleriyle bitir.

8. Koddan Bağlanma

8.1 .NET (Microsoft.Data.SqlClient)

Native vektör için **6.1.0+** şart. Strict encryption (TDS 8.0) ile bağlantı dizesi:

» 24-connection.cs

```
var cs = "Server=tcp:localhost,1433;Database=DevBook;User Id=app;Password=***;"
        + "Encrypt=Strict;TrustServerCertificate=False;";
await using var conn = new SqlConnection(cs);
await conn.OpenAsync();

// Vektör parametresi (Microsoft.Data.SqlClient 6.1+ / .NET 10 SqlDbType.Vector)
var cmd = new SqlCommand(
    "SELECT TOP 5 id, title FROM dbo.Articles ORDER BY VECTOR_DISTANCE('cosine', embedding, @q)", conn);
cmd.Parameters.Add(new SqlParameter("@q", new SqlVector<float>(queryEmbedding)));
await using var r = await cmd.ExecuteReaderAsync();
```

8.2 EF Core 10: semantik arama

» 25-efcore-search.cs

```
var q = new SqlVector<float>(queryEmbedding);
var results = await db.Articles
    .Select(a => new {
        a.Title,
        Distance = EF.Functions.VectorDistance("cosine", a.Embedding, q) })
    .OrderBy(x => x.Distance)
    .Take(10)
    .ToListAsync();
```

8.3 Python (pyodbc)

» 26-python.py

```
import pyodbc, json
cn = pyodbc.connect(
    "DRIVER={ODBC Driver 18 for SQL Server};SERVER=localhost;DATABASE=DevBook;"
    "UID=app;PWD=***;Encrypt=yes;TrustServerCertificate=yes")
cur = cn.cursor()
qv = json.dumps([0.3, 0.3, 0.3, 0.3, 0.3]) # vektörü JSON dizisi olarak gönder
cur.execute("""
    SELECT TOP (3) id, title, VECTOR_DISTANCE('cosine', embedding, CAST(? AS vector(5))) AS
    dist
    FROM dbo.Articles ORDER BY dist""", qv)
for row in cur.fetchall():
    print(row.id, row.title, row.dist)
```

İpucu: ODBC/JDBC gibi native vektörü henüz bilmeyen sürücülerde vektörü **JSON dizisi metni** olarak gönderip `CAST(... AS vector(n))` ile dönüştür, SQL Server geriye uyumluluk için vektörü metin olarak da kabul eder.

9. Yükseltme ve Uyumluluk (Developer Kontrol Listesi)

- **Uyumluluk düzeyi 170:** Bazı yeni fonksiyonlar (`AI_GENERATE_CHUNKS` vb.) bunu ister.
- **Kaldırılan bileşenler (discontinued):** Data Quality Services, Master Data Services, **Synapse Link** (→ **Fabric Mirroring**), Purview access policies, **Web edition**. Bunlara bağımlı kod/iş akışın varsa göç planla.
- **Kullanımdan kalkan (deprecated):** hot add CPU, lightweight pooling / fiber mode.
- **Sürücü sürümleri:** Native vektör için `Microsoft.Data.SqlClient` 6.1+, EF Core 10, .NET 10 (`SqlDbType.Vector`).
- **Önizleme** → **GA:** Vektör indeks / `VECTOR_SEARCH`, CES, fuzzy eşleşme kutu üründe önizleme; üretim kararını GA takvimine bağla, `PREVIEW_FEATURES` gerektiğini unutma.
- **Kutu vs bulut:** DiskANN "en güncel sürüm" indeks + `WITH APPROXIMATE` şu an yalnız Azure SQL DB / Fabric SQL; kutuda `VECTOR_SEARCH ... TOP_N` kullan.
- **Ücretsiz geliştirme SKU'ları:** Standard Developer ve Enterprise Developer, tam özellikli, üretim-dışı lisans.

10. Sık Karşılaşılan Hatalar ve Sorun Giderme

Bu tablo, kendi testlerimde **gerçekten karşılaştığım** hataları ve çözümlerini derler, vektör/AI özellikleriyle çalışırken en çok vakit kaybettiren noktalar.

Hata (mesaj)	Neden	Çözüm
Msg 1934 ... CREATE VECTOR INDEX failed ... 'QUOTED_IDENTIFIER'	Oturumda QUOTED_IDENTIFIER kapalı (sqlcmd varsayılını)	İndeksten önce SET QUOTED_IDENTIFIER ON;
Msg 102 ... Incorrect syntax near 'APPROXIMATE'	WITH APPROXIMATE kutu üründe yok	Kutuda VECTOR_SEARCH ... TOP_N kullan; WITH APPROXIMATE yalnız Azure SQL DB/Fabric
Msg 102 ... Incorrect syntax near '='	REGEXP_LIKE(...) = 0 , yüklem skaler gibi kullanılmış	WHERE NOT REGEXP_LIKE(...) ya da CASE WHEN REGEXP_LIKE(...)
Msg 207 ... Invalid column name 'id' (VECTOR_SEARCH)	Tablo kolonuna sonuç alias'ıyla erişildi	Tablo kolonları tablo alias'ı , distance sonuç alias'ı
Msg 42227 ... Cannot find a vector index with metric 'cosine'	İndeks yok ya da metriği uyuşmuyor	Aynı metrikle CREATE VECTOR INDEX ... WITH (METRIC='cosine', ...)
Incorrect syntax near 'REGEXP_LIKE' / fonksiyon bulunamıyor	Önizleme/uyumluluk bağlamı eksik	PREVIEW_FEATURES = ON + COMPATIBILITY_LEVEL = 170
AI_GENERATE_CHUNKS bulunamıyor	Uyumluluk düzeyi < 170	ALTER DATABASE ... SET COMPATIBILITY_LEVEL = 170;
Vektör indeks/arama "yokmuş" gibi davranıyor	PREVIEW_FEATURES kapalı	ALTER DATABASE SCOPED CONFIGURATION SET PREVIEW_FEATURES = ON;

Altın kural: Vektör/AI ile çalışan bir veritabanında oturumun başında üçlüyü garanti et, PREVIEW_FEATURES = ON , COMPATIBILITY_LEVEL = 170 , SET QUOTED_IDENTIFIER ON .

11. Pratik Tarifler (Cookbook)

Aşağıdaki tarifleri bizzat çalıştırıp doğruladım.

11.1 Hibrit arama: vektör + anahtar kelime

Semantik yakınlığı klasik bir filtreyle birleştir, önce WHERE süzer, sonra vektör mesafesi sıralar:

» r11-hybrid.sql

```
DECLARE @qv vector(5) = '[0.3,0.3,0.3,0.3,0.3]';
SELECT TOP (3) id, title, VECTOR_DISTANCE('cosine', embedding, @qv) AS dist
FROM dbo.Articles
WHERE title LIKE '%1%'           -- anahtar kelime / kategori filtresi
ORDER BY dist;                  -- semantik sıralama
```

11.2 JSON denetim günlüğü (audit log)

Değişiklikleri şemasız `json` olarak sakla, sorguyla analiz et:

» r11-audit.sql

```
CREATE TABLE dbo.Audit (Id int IDENTITY, Event json);
INSERT dbo.Audit (Event) VALUES
(N'{"user":"ada","action":"login","ok":true}'),
(N'{"user":"ada","action":"delete","ok":false}');

SELECT JSON_VALUE(Event,'$.user') AS usr, COUNT(*) AS silme_sayisi
FROM dbo.Audit
WHERE JSON_VALUE(Event,'$.action') = 'delete'
GROUP BY JSON_VALUE(Event,'$.user');
```

↳ gerçek çıktı / real output

```
usr  silme_sayisi
---  -
ada  1
```

11.3 Veri doğrulamayı tabloya göm: RegEx CHECK kısıtı

Geçersiz veriyi uygulamaya güvenmeden, motor düzeyinde reddet:

» r11-check.sql

```
CREATE TABLE dbo.Signup (
    email nvarchar(200) CHECK (REGEXP_LIKE(email, '^[\\w.+]+@[\\w-]+\\. [\\w.-]+$'))
);
INSERT dbo.Signup VALUES ('ok@dmc.com');    -- kabul
INSERT dbo.Signup VALUES ('bad@@');        -- CHECK ihlali → reddedilir
```

REGEXP_LIKE bir yüklem olduğu için CHECK kısıtında doğrudan kullanılabilir, uygulama katmanı atlansa bile geçersiz e-posta tabloya giremez.

11.4 Uçtan uca mini-proje: gerçek semantik arama

Bu örnekleri SQL Server 2025 üzerinde bizzat çalıştırdım: gerçek belgeleri yerel bir embedding modeliyle (Ollama `all-minilm`, 384 boyut) vektöre çevirdim, `vector(384)` kolonuna yükledim ve canlı

`VECTOR_DISTANCE` ile aradım. (Üretimde embedding için `AI_GENERATE_EMBEDDINGS` + Azure OpenAI kullanırsın, bkz. Bölüm 2.4; motorun HTTPS zorunluluğu unutulmamalı.)

1) Bilgi tabanı, gerçek embedding'lerle yükle. Her belge dış bir modelle embed edilir; sonuç `vector(384)` olarak saklanır:

» proje-1.sql

```
CREATE TABLE dbo.Kb (id int PRIMARY KEY, chunk nvarchar(300), embedding vector(384));
-- Her satır, bir embedding modelinden gelen 384 boyutlu gerçek vektörle doldurulur
INSERT dbo.Kb VALUES (1, N'SQL Server 2025 offers a built-in vector type and DiskANN index for semantic similarity search.', CAST('[...]' AS vector(384)));
-- ... 6 belge ...
```

2) Soruyu embed et, en yakın 3 belgeyi getir (canlı çıktı):

» proje-2.sql

```
DECLARE @q vector(384) = CAST('[...soru embedding...]' AS vector(384));
SELECT TOP (3) id, chunk, ROUND(VECTOR_DISTANCE('cosine', embedding, @q), 4) AS dist
FROM dbo.Kb ORDER BY dist;
```

Soru: "how do I find semantically similar text in the database?"

↳ gerçek çıktı / real output

```
id  chunk
dist
--  -----
-----
1   SQL Server 2025 offers a built-in vector type and DiskANN index for semantic similarity
search.    0.4082
3   The native JSON type stores documents in a binary format and speeds up indexed queries.
0.7696
4   Regular expression functions provide pattern matching and validation without CLR.
0.8228
```

Anlamsal arama çalışıyor: soru "vektör" kelimesini içermese de en yakın belge (0.41) **semantik aramadan** bahseden #1; JSON (0.77) ve RegEx (0.82) belirgin biçimde uzak. Mesafe farkı sıralamanın gerçekten anlama dayandığını gösterir.

3) API olarak sun, tek satır kod bile yazmadan. Aynı tabloyu Data API Builder ile REST'e aç (bkz. Bölüm 5.2); bu da **canlı** doğrulandı:

↳ GET /api/Article?\$first=2: DAB gerçek yanıtı

```
{ "value": [
  { "id": 1, "title": "Article 1", "embedding": [0.01, 0.02, 0.03, 0.04, 0.05] },
  { "id": 2, "title": "Article 2", "embedding": [0.02, 0.04, 0.06, 0.08, 0.10] } ] }
```

Sonuç: **chunk (canlı) → embed (gerçek model) → sakla (vector) → ara (VECTOR_DISTANCE, canlı) → sun (DAB REST, canlı)**, hepsi veritabanı merkezli, ayrı bir vektör DB'si veya arama servisi olmadan.

12. SQL Server 2022 → 2025: Developer Farkları

Konu	SQL Server 2022	SQL Server 2025
Vektör arama	Yok (harici vektör DB)	Yerleşik <code>vector</code> tipi + <code>VECTOR_SEARCH</code> + DiskANN
Embedding üretimi	Uygulama katmanında	T-SQL'de <code>AI_GENERATE_EMBEDDINGS</code> / <code>EXTERNAL MODEL</code>
JSON	<code>nvarchar(max)</code> + fonksiyonlar	Gerçek <code>json</code> tipi (ikili, indekslenebilir)
Desen eşleştirme	<code>LIKE</code> / CLR	Yerleşik <code>REGEXP_*</code> fonksiyonları
Eşzamanlılık	Klasik kilitleme	Optimized Locking (LAQ + TID)
API üretimi	Elle / harici	Data API Builder (otomatik REST/GraphQL)
Değişiklik akışı	CDC / CT	Change Event Streaming → Event Hubs
AI ajanı erişimi	Yok	SQL MCP Server
.NET vektör	Yok	<code>Microsoft.Data.SqlClient</code> 6.1 <code>SqlVector<float></code> , EF Core 10
Uyumluluk düzeyi	160	170

Ekler

Hızlı başvuru: sözlük, birincil kaynaklar ve yazar.

13. Sözlük

- **Embedding**: Bir metnin/görselin anlamını temsil eden sabit boyutlu sayı dizisi (vektör).
 - **Vektör (vector(n))**: n boyutlu float32 dizisi; benzerlik aramasının temeli.
 - **kNN (exact)**: Tüm satırlarla karşılaştırıp en yakın k komşuyu bulan **kesin** arama.
 - **ANN (approximate)**: Bir indeks (DiskANN) üzerinden **yaklaşık**, çok daha hızlı en yakın komşu araması.
 - **DiskANN**: Diske/ölçeğe uygun grafik tabanlı yaklaşık en yakın komşu indeks algoritması.
 - **RAG**: Retrieval-Augmented Generation; ilgili parçaları getirip LLM'e bağlam olarak verme.
 - **Cosine / Euclidean / Dot**: `VECTOR_DISTANCE` 'in desteklediği mesafe metrikleri.
 - **LAQ**: Lock After Qualification; satırı ancak niteledikten sonra kilitleyen Optimized Locking mekanizması.
 - **ADR / RCSI**: Accelerated Database Recovery / Read Committed Snapshot Isolation; Optimized Locking'in ön koşulları.
 - **MCP**: Model Context Protocol; AI ajanlarının araç/veri kaynaklarına standart erişimi.
 - **CES**: Change Event Streaming; DML değişikliklerini Event Hubs'a CloudEvent olarak yayınlama.
 - **DAB**: Data API Builder; tablo/view'lardan otomatik REST/GraphQL üreten araç.
 - **TDS 8.0**: Tabular Data Stream 8; TLS 1.3 ile şifreli SQL Server iletişim protokolü.
 - **PREVIEW_FEATURES**: Önizleme özelliklerini açan veritabanı-kapsamlı yapılandırma.
-

14. Kaynakça

Tüm özellik ve fonksiyon adları ile GA / önizleme durumları aşağıdaki birincil Microsoft kaynaklarından doğrulanmış; örnekler SQL Server 2025 (RTM-CU7, 17.0.4065.4) üzerinde çalıştırılmıştır.

- Microsoft Learn, What's New in SQL Server 2025 (17.x): <https://learn.microsoft.com/en-us/sql/sql-server/what-s-new-in-sql-server-2025>
- CREATE EXTERNAL MODEL (T-SQL): <https://learn.microsoft.com/en-us/sql/t-sql/statements/create-external-model-transact-sql>
- AI_GENERATE_EMBEDDINGS (T-SQL): <https://learn.microsoft.com/en-us/sql/t-sql/functions/ai-generate-embeddings-transact-sql>
- AI_GENERATE_CHUNKS (T-SQL): <https://learn.microsoft.com/en-us/sql/t-sql/functions/ai-generate-chunks-transact-sql>
- VECTOR_SEARCH (T-SQL): <https://learn.microsoft.com/en-us/sql/t-sql/functions/vector-search-transact-sql>
- vector veri tipi (T-SQL): <https://learn.microsoft.com/en-us/sql/t-sql/data-types/vector-data-type>

- SqlClient'te vektör desteği (ADO.NET): <https://learn.microsoft.com/en-us/sql/connect/ado-net/sql/vector-data-sql-server>
- EF Core, SQL Server Vector Search: <https://learn.microsoft.com/en-us/ef/core/providers/sql-server/vector-search>
- SqlVector ile EF Core ve Dapper (Azure SQL Dev Corner): <https://devblogs.microsoft.com/azure-sql/using-the-new-sqlvector-type-with-ef-core-and-dapper/>
- SQL Server 2025 vektör/AI (Azure SQL Dev Corner): <https://devblogs.microsoft.com/azure-sql/sql-server-2025-embraces-vectors-setting-the-foundation-for-empowering-your-data-with-ai/>
- JSON veri tipi (T-SQL): <https://learn.microsoft.com/en-us/sql/relational-databases/json/json-data-type>
- Optimized locking: <https://learn.microsoft.com/en-us/sql/relational-databases/performance/optimized-locking>
- Data API Builder: <https://learn.microsoft.com/en-us/azure/data-api-builder/overview>
- Change event streaming: <https://learn.microsoft.com/en-us/sql/relational-databases/track-changes/change-event-streaming/overview>
- SQL Server 2025 Docker imajı: <https://mcr.microsoft.com/en-us/product/mssql/server/about>

Yazar Hakkında



Çağlar Özenç: Kurucu, Microsoft Data Platform MVP

Çağlar Özenç, Kurucu ve **Microsoft Data Platform MVP**.

Yıllardır SQL Server ve veri platformlarıyla uğraşıyorum: performans ayarlama, felaket kurtarma, yedekleme ve yüksek erişilebilirlik benim asıl işim. Kurucusu olduğum DMC Bilgi Teknolojileri, 15 yılı aşkın saha deneyimiyle kamu ve özel sektörde yüzlerce veri projesini hayata geçirdi; proaktif danışmanlık ve 7/24 destekle veritabanlarını güvenle yönetiyoruz. Bu kitabı da sahada öğrendiklerimi SQL Server 2025 üzerinde bizzat sınavarak yazdım.

Kişisel web: caglarozenc.com

İletişim: dmcteknoloji.com · +90 212 945 61 66 · info@dmcteknoloji.com

Örnekleri SQL Server 2025 üzerinde bizzat çalıştırıp doğruladım; kesin sözdizimini kurulu derlemene ve resmi dokümana karşı teyit et, önizleme özellikleri GA'ya kadar değişebilir.